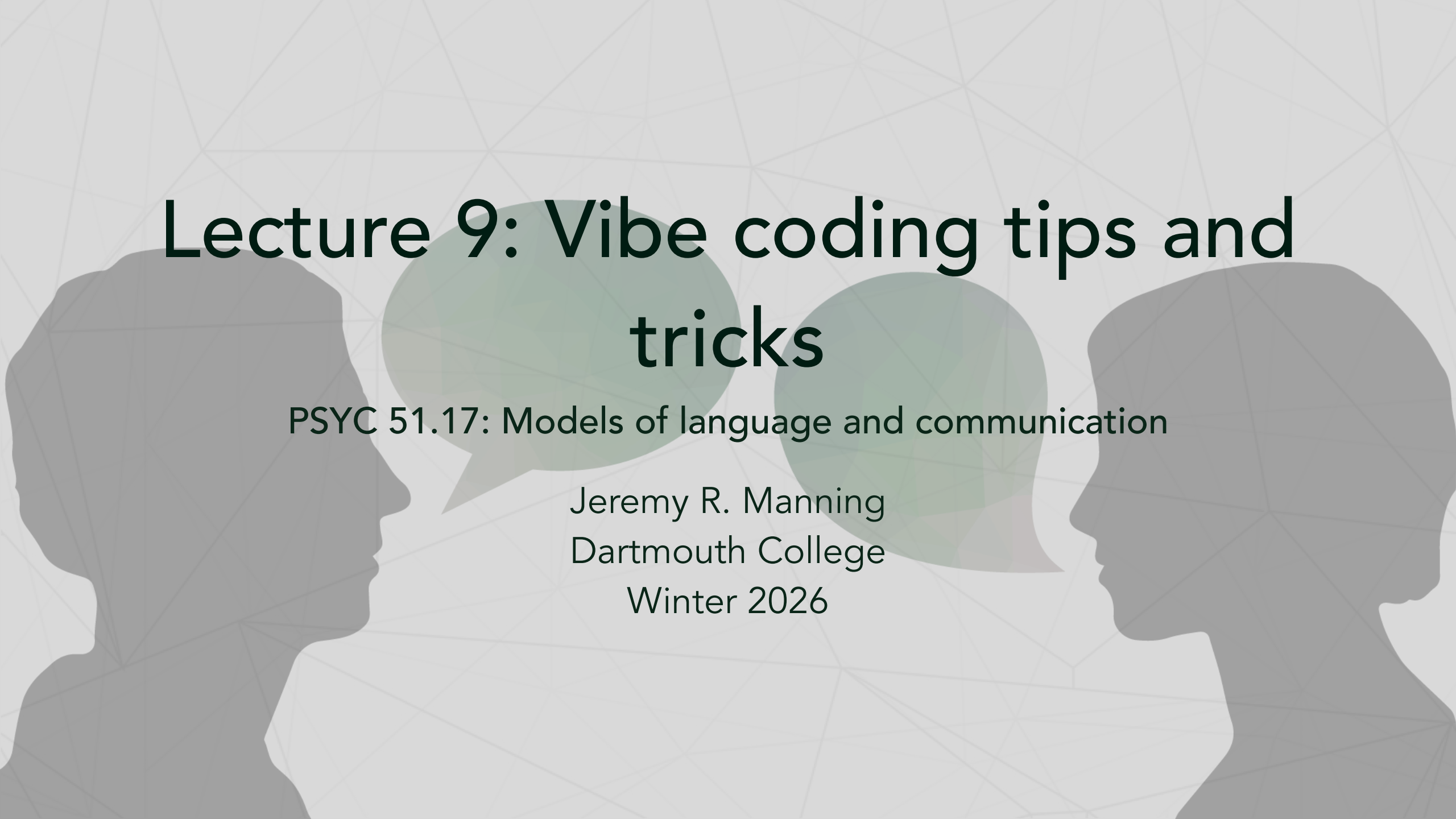
Lecture 9: Vibe coding tips and tricks

PSYC 51.17: Models of language and communication

Jeremy R. Manning
Dartmouth College
Winter 2026



Today's agenda

1-Hour X-Hour Tutorial

1. **Free AI tools for students** (5 min) - GitHub Copilot, Google Gemini
2. **Setting up your environment** (10 min) - VS Code, OpenCode, oh-my-opencode
3. **The spec-kit workflow** (15 min) - From design docs to implementation
4. **Live demo** (25 min) - Build something together!
5. **Q&A** (5 min)

Hands-on session

Follow along! Open your terminal and VS Code.

Free AI coding tools for students

GitHub Copilot (FREE)

- AI pair programmer in VS Code
- Autocomplete + chat assistance
- Sign up: github.com/education/students

Google Gemini (1 year FREE)

- Gemini 2.5/3 Pro models
- 2TB storage included
- Sign up: gemini.google/students

Other options

- **Cursor** - AI-first IDE (cursor.com/students)
- **Codeium/Windsurf** - Free forever tier
- **Dartmouth GenAI** - chat.dartmouth.edu

Recommendation

Start with **GitHub Copilot** + **OpenCode** for the best terminal + IDE combo.

What is "vibe coding"?

Vibe Coding

Using AI coding agents to rapidly prototype and implement software by describing what you want in natural language, then iterating on the output.

The old way

1. Think about the problem
2. Research documentation
3. Write code line by line
4. Debug for hours
5. Repeat

The vibe coding way

1. Describe what you want clearly
2. Let AI draft the solution
3. Review and iterate
4. Verify it works
5. Ship it!

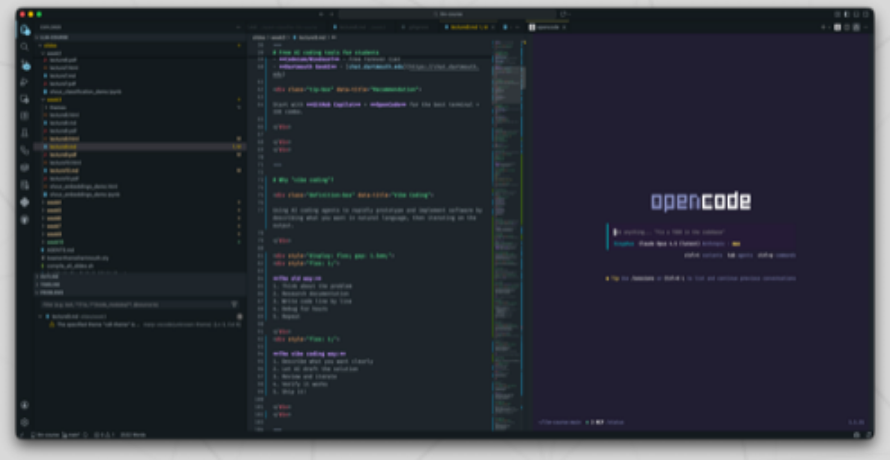
Installing VS Code

Download and install

1. Go to code.visualstudio.com
2. Download for your OS (Mac/Windows/Linux)
3. Run the installer
4. Launch VS Code

Essential extensions

- GitHub Copilot
- Python
- Jupyter



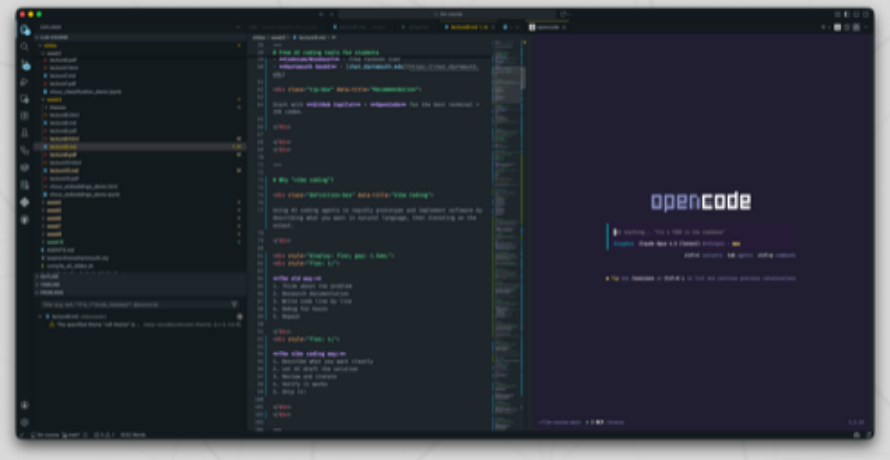
Activating GitHub Copilot

Setup steps

1. Click the **Accounts** icon (bottom left of VS Code)
2. Sign in with your GitHub account
3. Copilot activates automatically!

Verify it works

Type a comment like `# function to calculate fibonacci` and watch Copilot suggest code!



Your terminal

The terminal is where we'll run OpenCode. Open it from VS Code (View → Terminal) or use your system terminal.

Terminal basics

- **Mac**: Terminal.app or iTerm2
- **Windows**: PowerShell or Windows Terminal
- **Linux**: Your preferred terminal

Installing OpenCode

OpenCode is a terminal-based AI coding agent. Think ChatGPT, but it can read and write your files!

Installation (choose one)

```
curl -fsSL https://opencode.ai/install | bash   # One-liner (recommended)
npm install -g opencode-ai                      # npm
brew install anomalyco/tap/opencode             # Homebrew (Mac)
```

Connect your API key

```
opencode auth login                             # Interactive setup
export ANTHROPIC_API_KEY="your-key"             # Or set environment variable
```

Starting OpenCode

Launch OpenCode

```
cd ~/my-project
opencode
```

First command

Type `/init` inside OpenCode to create an `AGENTS.md` file that helps the AI understand your project!

OpenCode in action

Try it out

Ask OpenCode to do something:

"Create a Python function that reverses a string"

Watch it write code, create files, and run tests!

Installing oh-my-opencode

oh-my-opencode supercharges OpenCode with specialized agents and power features.

Installation

```
bunx oh-my-opencode install    # Recommended
npx oh-my-opencode install     # Alternative
```

What it adds

- **Sisyphus** - Primary orchestrator (plans and delegates)
- **Oracle** - Architecture and code review expert (GPT-5.2)
- **Librarian** - Documentation and research
- **Explore** - Fast codebase navigation

Ultrawork mode

Ultrawork mode transforms the AI from a chatbot into an autonomous software engineer.

Key features

- Maximum precision execution
- Parallel agent orchestration
- Mandatory verification (nothing is "done" without proof)
- Zero tolerance for partial completion

The philosophy

"Claim nothing without proof. Execute. Verify. Show evidence."

Ultrawork mode ensures the AI doesn't stop at 80% - it finishes 100%.

The /ralph-loop command

`/ralph-loop` is a self-referential development loop that runs until task completion.

How it works

```
/ralph-loop "Build a REST API with authentication"
/ralph-loop "Refactor the payment module" --max-iterations=50
```

The agent keeps working until it outputs `<promise>DONE</promise>` or hits the iteration limit.

Use case

Perfect for large refactors or when you want the AI to "ship code overnight" while you're away!

Cancel anytime with `/cancel-ralph`.

The 4-step vibe coding workflow

1. Detailed description

- What are the inputs/outputs?
- What are the edge cases?
- Include examples!

2. Technical design doc

- Have AI draft the architecture
- Iterate until you're happy
- Include skeleton code

3. Implementation plan

- Break into small tasks
- Include verification steps
- Don't exceed context limits

4. Implement and verify

- Let AI write the code
- Test each piece
- Stress test the result!

Introducing spec-kit

spec-kit is GitHub's toolkit for Spec-Driven Development (SDD).

Core idea

Instead of "vibe coding" a prompt, you write a **specification** first. The spec becomes the executable source of truth.

The workflow

1. `/speckit.specify` - Create the spec (what + why, no how)
2. `/speckit.clarify` - AI asks clarifying questions
3. `/speckit.plan` - Generate technical plan
4. `/speckit.tasks` - Break into actionable tasks
5. `/speckit.implement` - Execute with verification

Writing a good spec

The golden rule

Focus on **WHAT** and **WHY**, not HOW. No languages, databases, or APIs in the spec!

Example spec structure

```
### User Story 1 - Add task to board (Priority: P1)
User can create a new task with a title and optional description.
Task appears in the "To Do" column by default.

**Acceptance Scenarios:**
1. **Given** an empty board, **When** user clicks "Add Task"
   **Then** a form appears with title and description fields
2. **Given** a filled form, **When** user clicks "Save"
   **Then** task appears in "To Do" column
```

Demo: AI-Powered Search Engine

What we're building

A **single HTML file** that creates an AI-powered search engine:

1. User asks a question
2. Browser-based LLM (SmolLM2-360M) generates search terms
3. Fetches and parses Google results
4. LLM summarizes each result, then synthesizes a final answer
5. Returns answer with annotated references

The impressive part

Minimalist UI with smooth animations + real-time progress showing which pipeline step is running and estimated time remaining!

The spec-kit workflow for our demo

Key difference from ad-hoc prompting

Each step produces a **document** that becomes the source of truth. The AI can't drift from the spec!

Step 1: Constitution

The constitution defines **architectural DNA** - rules that govern ALL code.

Prompt: /speckit.constitution

```
Create a constitution for this project with these principles:
- Single HTML file (no build tools, no npm)
- Use Transformers.js for browser-based LLM inference
- Model: SmolLM2-360M-Instruct (small enough for browser)
- Modern CSS (flexbox, grid, CSS variables)
- Vanilla JavaScript only (no frameworks)
- Graceful degradation if WebGPU unavailable
```

Output

A `constitution.md` file with inviolable rules.

Step 2: Specify

Now we describe WHAT we want (not HOW).

Prompt: /speckit.specify

```
Build a web-based AI search assistant. Single HTML file.

User Journey:
1. User sees "Ask me a question!" prompt
2. User types question and submits
3. System shows real-time progress with current step and time estimate
4. System displays final answer with numbered references

Non-functional requirements:
- Loads in <3 seconds on 4G
```

continued...

Step 2: Specify

```
11 - Works offline after first load (model cached)
12 - Minimalist, modern aesthetic
```

...continued

Step 3: Clarify

The AI identifies ambiguities and asks questions.

AI will ask things like

1. How should search results be fetched? (CORS issues with Google)
2. What if the LLM generates poor search terms?
3. How many results to fetch? (You said 10)
4. What's the fallback if WebGPU is unavailable?
5. Should users be able to ask follow-up questions?

Our answers

Use a CORS proxy for Google. Fetch 10 results. Fall back to WASM. No follow-ups for MVP.

Your answers become part of the spec!

Step 4: Plan

Now we specify the technical approach.

Prompt: /speckit.plan

```
Create a technical plan using:
- Transformers.js with SmolLM2-360M-Instruct
- Google search via allorigins.win CORS proxy
- Single HTML file with embedded CSS/JS
- CSS animations for progress states
- LocalStorage for model caching

Include: architecture diagram, data flow, component breakdown.
```

Output

`plan.md` with architecture, data models, component specs.

The pipeline architecture

LLM Call 1: Query to keywords

"What causes the northern lights?" → "aurora borealis cause science"

LLM Calls 2-12: Process results

Summarize each of 10 results, then synthesize final answer with citations.

Step 5: Tasks

Break into implementable chunks.

Prompt: /speckit.tasks

```
Break the plan into tasks. Each task should:
- Be completable in <30 minutes
- Have clear acceptance criteria
- Be independently testable
```

Generated tasks

1. HTML skeleton + CSS variables + loading animation
2. Transformers.js setup + model loading with progress
3. Search term generation prompt + LLM call
4. Google search fetch + result parsing
5. Result summarization loop with progress updates
6. Final synthesis + reference formatting
7. Error handling + offline support

Step 6: Implement - Task 1

Prompt for Task 1

```
Implement Task 1: HTML skeleton with CSS.

Requirements from spec:
- Single input field with "Ask me a question!" placeholder
- Submit button (disabled during processing)
- Progress area showing: current step, progress bar, time estimate
- Results area with answer + references
- CSS: dark mode, smooth transitions, modern sans-serif font
- Mobile responsive

Write the complete HTML file with embedded <style>.
```

Step 6: Implement - Task 2

Prompt for Task 2

```
Implement Task 2: Add Transformers.js and model loading.

Requirements:
- Import Transformers.js from CDN
- Load SmolLM2-360M-Instruct on page load
- Show model download progress (it's ~400MB)
- Cache model in browser storage
- Update UI: "Loading AI model... 45%"
- Handle WebGPU vs WASM fallback

Add to the existing HTML file.
```

Step 6: Implement - Task 3

Prompt for Task 3

```
Implement Task 3: Search term generation.

Requirements:
- Take user question as input
- Prompt SmolLM2 to extract 3-5 search keywords
- Prompt template: "Extract search keywords from this question.
  Return only keywords, comma-separated. Question: {question}"
- Parse response, handle edge cases
- Update progress: "Generating search terms..."

Add to the existing HTML file.
```

Step 6: Implement - Tasks 4-5

Task 4: Google Search

Fetch via CORS proxy, parse HTML for titles, snippets, and URLs. Handle rate limits gracefully.

Task 5: Summarization

For each of 10 results: call SmolLM2 with snippet, update progress ("Summarizing 3/10..."), store summary with source URL.

Step 6: Implement - Tasks 6-7

Task 6: Final Synthesis

Combine all summaries + question. Generate comprehensive answer. Format with [1], [2] citations. Display with clickable references.

Task 7: Error Handling

Network failures → retry with backoff. Model errors → show friendly message. Timeout → partial results OK.

The final result

User experience

1. Clean input: "Ask me a question!"
2. Types: "Why is the sky blue?"
3. Progress bar animates through steps
4. Answer appears with smooth fade-in
5. References listed below with summaries

Under the hood

- ~400MB model cached locally
- 12 LLM inference calls
- ~45 seconds total (first run)
- ~30 seconds (cached model)
- Works offline after caching!

Single HTML file! No server. No API keys.

Why this demo matters

Technical skills

- Browser-based ML inference
- Async JavaScript pipelines
- Progress UI patterns
- CORS and web scraping

Vibe coding skills

- Constitution → constraints
- Spec → requirements
- Plan → architecture
- Tasks → incremental progress

The meta-lesson

We just designed a complex app without writing a single line of code ourselves. The spec-kit workflow gave us a **complete, implementable plan**.

Testing the search engine

Manual testing

- Open file in browser
- Check model loads (watch progress)
- Try simple question
- Try edge cases: empty query, very long query, non-English

Verify each pipeline step

1. Search terms make sense?
2. Google results fetched?
3. Summaries are coherent?
4. Final answer cites sources?
5. References link correctly?

Don't skip this!

Open DevTools Console. Watch for errors. LLM output can be unpredictable!

Common pitfalls

Context overflow

- Break tasks into smaller chunks
- Don't paste entire codebases
- Use `/compact` in OpenCode

Hallucinations

- AI invents plausible but wrong APIs
- Always verify against real docs
- Test early and often

Vague prompts

- "Make it better" → fails
- "Add input validation for edge case X" → works

Rule of thumb

If you can't verify it, don't accept it.

Best practices summary

The vibe coding checklist

1. **Describe clearly** - Include inputs, outputs, edge cases, examples
2. **Design first** - Get a technical design doc before coding
3. **Plan small** - Break into testable ~50-line tasks
4. **Verify everything** - Run tests, try it yourself, check edge cases
5. **Iterate** - Don't expect perfection on first try

Remember

You don't need to know HOW to code it, but you MUST know WHAT it should do!

Resources

Tools

- OpenCode
- oh-my-opencode
- spec-kit
- VS Code

Free for students

- GitHub Education
- Google Gemini

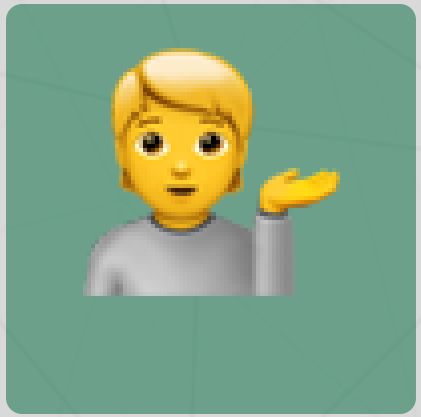
Course resources

- Assignment 1 vibe coding tips
- Dartmouth GenAI
- Course Discord for questions!

Questions?

Email me

Join our Discord



Come to office hours

Next up

Lecture 10: Classic embeddings (LSA, LDA) - Friday!